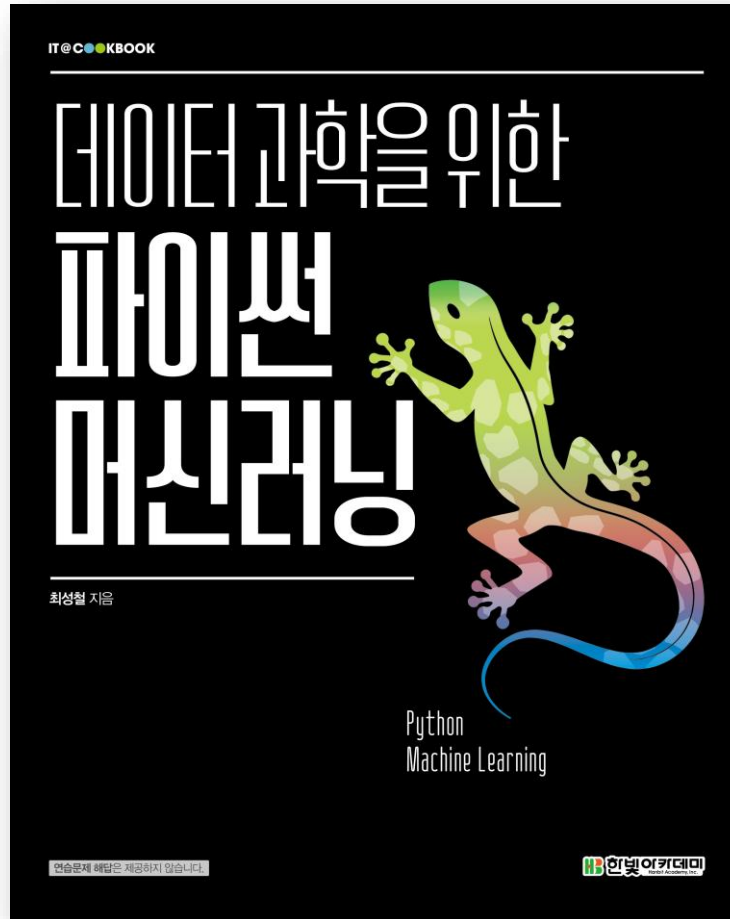




CHAPTER 07 선형회귀의 기초

CHAPTER 07

선형회귀의 기초

- 01 선형회귀의 이해
- 02 선형회귀의 기초 수식
- 03 최소자승법으로 선형회귀 풀기
- 04 경사하강법으로 선형회귀 풀기
- 05 선형회귀 성능 측정하기
- 06 코드로 선형회귀 구현하기

학습목표

- 선형회귀의 개념에 대해 이해한다.
- 선형회귀의 기초 수식을 이해하기 위해 비용함수의 개념과 비용함수의 편미분에 대해 학습한다.
- 최소자승법과 경사하강법으로 선형회귀를 풀어본다.
- 선형회귀 성능을 측정하기 위해 성능 측정 지표에 대해 알아본다.
- 코드로 선형회귀를 구현해본다

00

머신러닝의 이해

01 선형회귀의 이해

1. 선형회귀의 개념

- 선형회귀(Linear Regression) : 종속변수 y 와 한 개 이상의 독립변수 x 와의 선형 상관관계를 모델링하는 회귀 분석 기법
- 기존 데이터를 활용해 연속형 변수값을 예측
- $y = ax + b$ 수식을 만들고 a 와 b 의 값을 찾아냄

01 선형회귀의 이해

- 앞으로 개봉할 영화 예상 관객 수 y 를 예측하는 문제



그림 7-1 왓차 '보고싶어요' 수로 예상한 '옥자' 관객 수

01 선형회귀의 이해

- 실제 관측 수를 y 로 표현하여 좌표평면 상에 나타냄

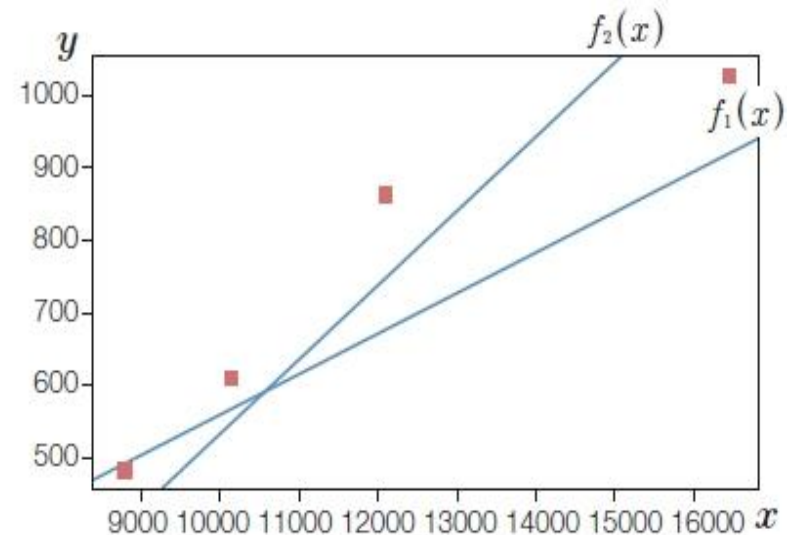
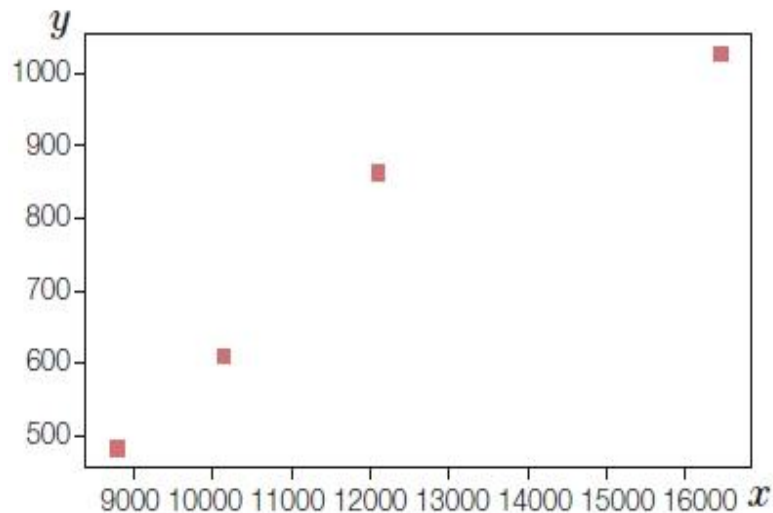


그림 7-2 좌표평면에 작성한 2개의 직선, $f_1(x)$ 와 $f_2(x)$

01 선형회귀의 이해

- 두 그래프 중 어떤 것이 기존 데이터를 ‘잘 표현하는가’
- 예측값이 실제값 대비 차이가 많이 나지 않는 그래프

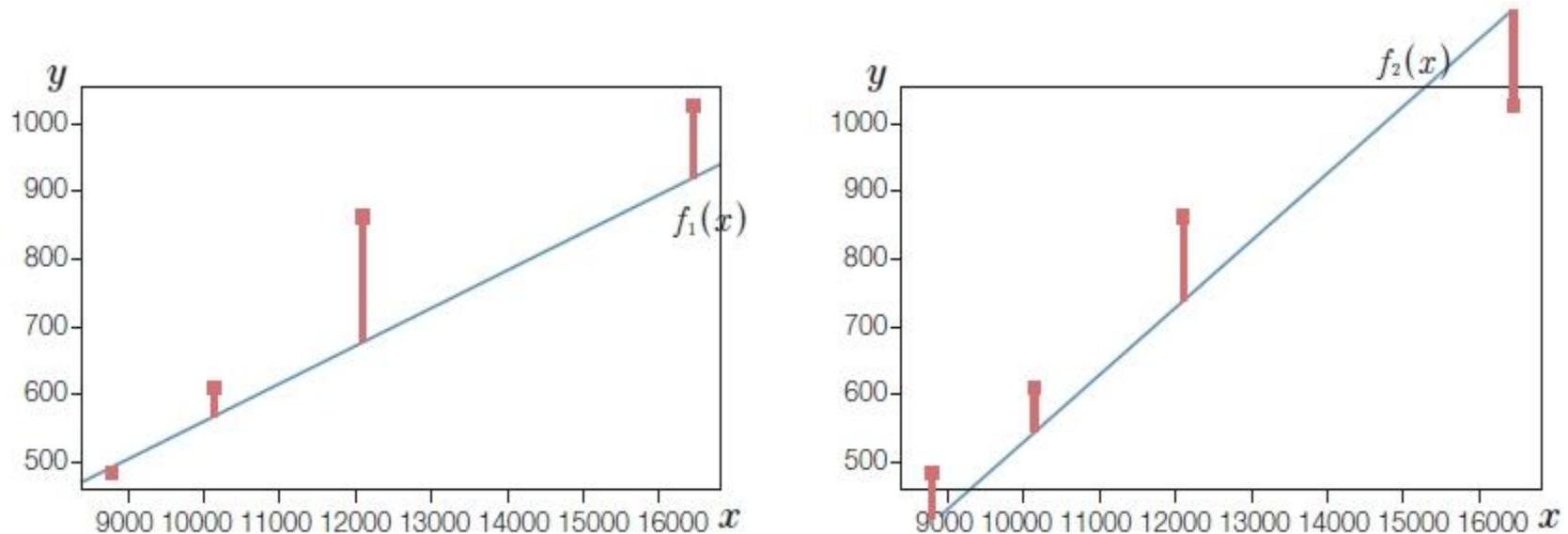


그림 7-3 $f_2(x)$ 가 $f_1(x)$ 보다 좀 더 데이터를 잘 표현하고 있다.

01 선형회귀의 이해

2. 예측 함수와 실제값 간의 차이

- 예측 함수는 예측값과 실제값 간의 차이를 최소화하는 방향으로
- 데이터 n 개 중 i 번째 데이터의 y 값에 대한 실제값과 예측값의 차이 $\hat{y}^i - y^i$
- 데이터가 5개 있을 때 5개 데이터의 오차의 합
$$(\hat{y}^{(1)} - y^{(1)}) + (\hat{y}^{(2)} - y^{(2)}) + (\hat{y}^{(3)} - y^{(3)}) + (\hat{y}^{(4)} - y^{(4)}) + (\hat{y}^{(5)} - y^{(5)})$$
 - 오차 값들이 음수와 양수로 나왔을 때 값들 간의 차이가 상쇄되어 0으로 계산될 수 있음

01 선형회귀의 이해

- 값의 제곱을 사용하여 오차의 합을 표현

$$(\hat{y}^{(1)} - y^{(1)})^2 + (\hat{y}^{(2)} - y^{(2)})^2 + (\hat{y}^{(3)} - y^{(3)})^2 + (\hat{y}^{(4)} - y^{(4)})^2 + (\hat{y}^{(5)} - y^{(5)})^2$$

$$\sum_{i=1}^n (\hat{y}^{(i)} - y^{(i)})^2$$

- 같은 식을
행렬로 표현

$$\hat{y} = \begin{bmatrix} w_1 \times 8759 + w_0 \\ w_1 \times 10132 + w_0 \\ w_1 \times 12078 + w_0 \\ w_1 \times 16430 + w_0 \end{bmatrix} \quad y = \begin{bmatrix} 487 \\ 612 \\ 866 \\ 1030 \end{bmatrix}$$

$$(\hat{y} - y)^2 = \begin{bmatrix} (w_1 \times 8759 + w_0 - 487)^2 \\ (w_1 \times 10132 + w_0 - 612)^2 \\ (w_1 \times 12078 + w_0 - 866)^2 \\ (w_1 \times 16430 + w_0 - 1030)^2 \end{bmatrix}$$

01 선형회귀의 이해

- 제곱 오차(square error) : $(\hat{y} - y)^2$ 로 예측값과 실제값의 제곱을 표시하여 오차를 나타냄
- 제곱 오차를 최소화하는 w_0 와 w_1 을 찾아야 함

$$\sum_{i=1}^n (w_i x^{(i)} + w_0 \times 1 - y^{(i)})^2$$

02

선형회귀의 기초 수식

02 선형회귀의 기초 수식

1. 비용함수의 개념

- **비용함수(cost function) : 머신러닝에서 최소화해야 할 예측값과 실제값의 차이**
- 가설함수(hypothesis function) : 예측값을 예측하는 함수

$$f(x) = h_{\theta}(x)$$

- 함수 입력값은 x 이고
함수에서 결정할 것은 θ 로, 가중치(weight) 값인 w_n

02 선형회귀의 기초 수식

- 비용함수가 두 개의 가중치 값으로 결정됨

$$J(w_0, w_1) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - (y^{(i)}))^2$$

- **잔차의 제곱합(Error sum of squares)** : 예측값인 가설함수와 실제값인 y 값 간의 차이를 제곱해서 모두 합함
 - 총 데이터는 m 개가 존재하고 각 데이터의 예측값과 실제값을 뺀 후 제곱한 값들을 모두 합한 값
- **손실함수(loss function)** : 비용함수에서 잔차의 제곱합 부분
- **평균 제곱 오차(mean squared error, MSE)** : 잔차의 제곱합을 $2m$ 으로 나눈 값

02 선형회귀의 기초 수식

- 비용함수를 수식의 목적이 나타나도록 표기

$$\arg \min_{\theta} \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

- 비용함수 그래프

- x, y축에는 w_0 와 w_1 값
z축에는 비용함수 값 J
- 가중치 값 설정에 따른 J 값을 볼 수 있음

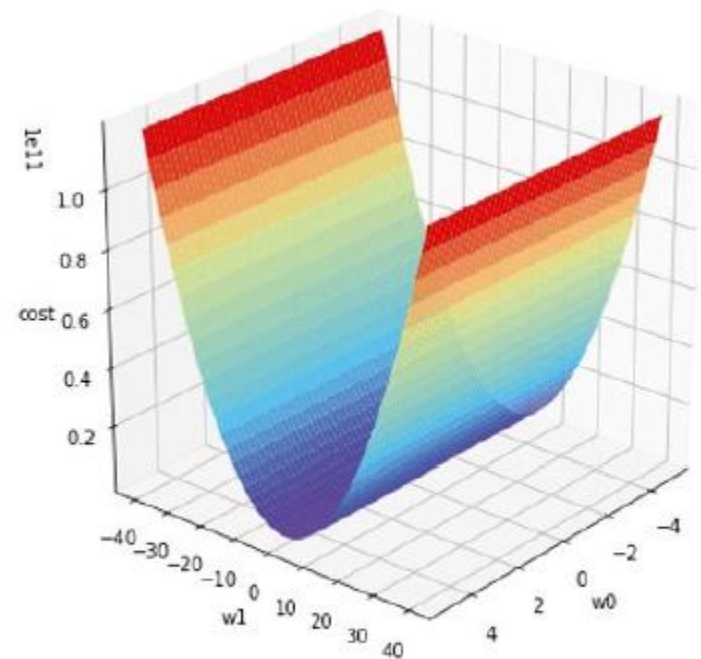


그림 7-4 비용함수 그래프

03

최소자승법으로 선형회귀 풀기

03 최소자승법으로 선형회귀 풀기

1. 최소자승법으로 풀기

- 최소자승법(least square method) : 선형대수의 표기법을 사용하여 방정식으로 선형회귀 문제를 푸는 방법

- 비용함수를 미분

$$J(w_0, w_1) = \frac{1}{2m} \sum_{i=1}^m (w_1 x^{(i)} + w_0 - y^{(i)})^2$$

$$\frac{\partial J}{\partial w_0} = \frac{1}{m} \sum_{i=1}^m (w_1 x^{(i)} + w_0 - y^{(i)})$$

$$\frac{\partial J}{\partial w_1} = \frac{1}{m} \sum_{i=1}^m (w_1 x^{(i)} + w_0 - y^{(i)}) x^{(i)}$$

03 최소자승법으로 선형회귀 풀기

- 미분한 값을 0으로 만드는 최적값 w_j^* 을 구한다

$$\frac{\partial J}{\partial w_0} = \frac{1}{m} \sum_{i=1}^m (w_1 x^{(i)} + w_0 - y^{(i)})$$

$$\frac{\partial J}{\partial w_1} = \frac{1}{m} \sum_{i=1}^m (w_1 x^{(i)} + w_0 - y^{(i)}) x^{(i)}$$

$$\hat{w}_0 m + \hat{w}_1 \sum x^{(i)} = \sum y^{(i)} \quad (1)$$

$$\hat{w}_0 \sum x^{(i)} + \hat{w}_1 \sum (x^{(i)})^2 = \sum y^{(i)} x^{(i)}$$

03 최소자승법으로 선형회귀 풀기

- 데이터가 5개만 있다고 가정하고 행렬로 표현

$$X = \begin{bmatrix} 1 & x^{(1)} \\ 1 & x^{(2)} \\ 1 & x^{(3)} \\ 1 & x^{(4)} \\ 1 & x^{(5)} \end{bmatrix} \quad \hat{w} = \begin{bmatrix} \hat{w}_0 \\ \hat{w}_1 \end{bmatrix} \quad y = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ y^{(3)} \\ y^{(4)} \\ y^{(5)} \end{bmatrix}$$

$$X^T X = \begin{bmatrix} m & \sum x^{(i)} \\ \sum x^{(i)} & \sum (x^{(i)})^2 \end{bmatrix}$$

- $(X^T X) \hat{w} = X^T y$ 이 (1)번 식과 같음

$$\hat{w} = (X^T X)^{-1} X^T y$$

03 최소자승법으로 선형회귀 풀기

$$X^T X = \begin{bmatrix} m & \sum x^{(i)} \\ \sum x^{(i)} & \sum (x^{(i)})^2 \end{bmatrix} = \begin{bmatrix} m & m\bar{x} \\ m\bar{x} & \sum (x^{(i)})^2 \end{bmatrix}$$

$$\begin{aligned} |X^T X| &= m \sum (x^{(i)})^2 - (m\bar{x})^2 \\ &= m \left(\sum (x^{(i)})^2 - m\bar{x}^2 \right) \\ &= m \left(\sum (x^{(i)} - \bar{x})^2 \right) \end{aligned}$$

$$\begin{aligned} (X^T X)^{-1} &= \frac{1}{m \sum (x^{(i)} - \bar{x})^2} \begin{bmatrix} \sum (x^{(i)})^2 & -m\bar{x} \\ -m\bar{x} & m \end{bmatrix} \\ &= \frac{1}{\sum (x^{(i)} - \bar{x})^2} \begin{bmatrix} \sum (x^{(i)})^2 / m & -\bar{x} \\ -\bar{x} & m \end{bmatrix} \end{aligned}$$

03 최소자승법으로 선형회귀 풀기

$$\begin{aligned}\hat{w} &= (X^T X)^{-1} X^T y = \begin{bmatrix} \hat{w}_0 \\ \hat{w}_1 \end{bmatrix} \\ &= \frac{1}{\sum (x^{(i)} - \bar{x})^2} \begin{bmatrix} \sum (x^{(i)})^2 / m & -\bar{x} \\ -\bar{x} & m \end{bmatrix} \begin{bmatrix} \sum y^{(i)} \\ \sum x^{(i)} y^{(i)} \end{bmatrix}\end{aligned}$$

$$\hat{w}_1 = \frac{\sum x^{(i)} y^{(i)} - m \bar{x} \bar{y}}{\sum (x^{(i)} - \bar{x})^2}$$

$$\hat{w}_0 = \bar{y} - \hat{w}_1 \bar{x}$$

- 변수가 늘어나면 $(X^T X)^{-1}$ 수식만 변경되어 행렬식만 구하면 되어 간단히 연산 가능

03 최소자승법으로 선형회귀 풀기

2. 최소자승법의 활용

- $X^T X$ 가 존재할 때 문제의 해를 구할 수 있음
- 최소자승법의 장점 : 반복(iteration)과 사용자가 지정하는 하이퍼 매개변수(hyper parameter)가 존재하지 않아서 데이터만 있으면 쉽게 해를 구할 수 있음
- 단점 : 피쳐가 늘어나면 속도가 느려짐
 - 현재 컴퓨터의 연산속도로는 다른 알고리즘에 비해 느린 것이 아님

최소자승법 실습 - 코랩 코드

수학적 의미:

기울기 w 는 다음 공식을 사용해 계산됩니다:

$$w = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}$$

- **numerator**: $\sum (x_i - \bar{x})(y_i - \bar{y}) \rightarrow$ 공분산 (x, y 의 연관성)
- **denominator**: $\sum (x_i - \bar{x})^2 \rightarrow x$ 의 분산

- **numerator** 는 x 와 y 가 **같이 얼마나 변하는지를 측정합니다** (공분산).
- **denominator** 는 x 가 **얼마나 흩어져 있는지, 즉 x 의 분산**입니다.
- 둘을 나누면 **y 가 x 에 따라 얼마나 변하는지, 즉 기울기**가 됩니다.

04

경사하강법으로 선형회귀 풀기

04 경사하강법으로 선형회귀 풀기

1. 경사하강법의 개념

- 경사하강법(gradient descent) : 경사를 하강하면서 수식을 최소화하는 매개변수의 값을 찾아내는 방법

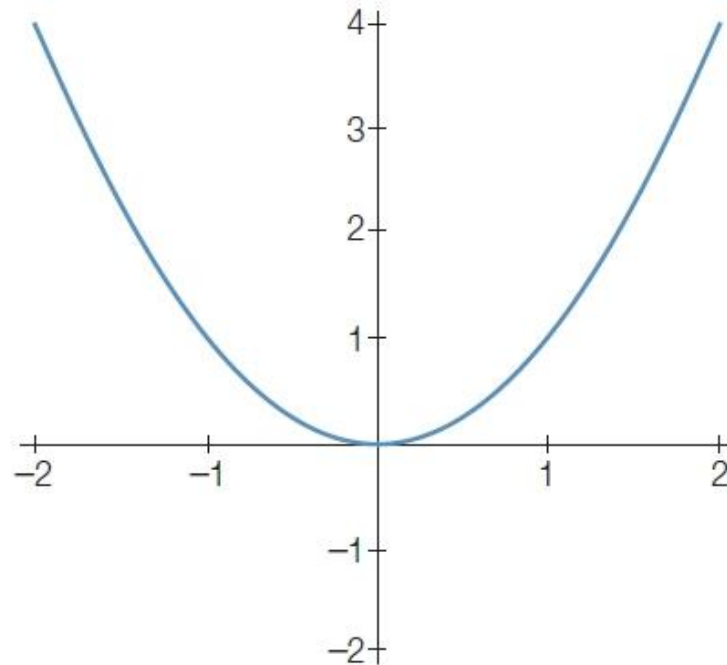


그림 7-5 $y = x^2$ 그래프

04 경사하강법으로 선형회귀 풀기

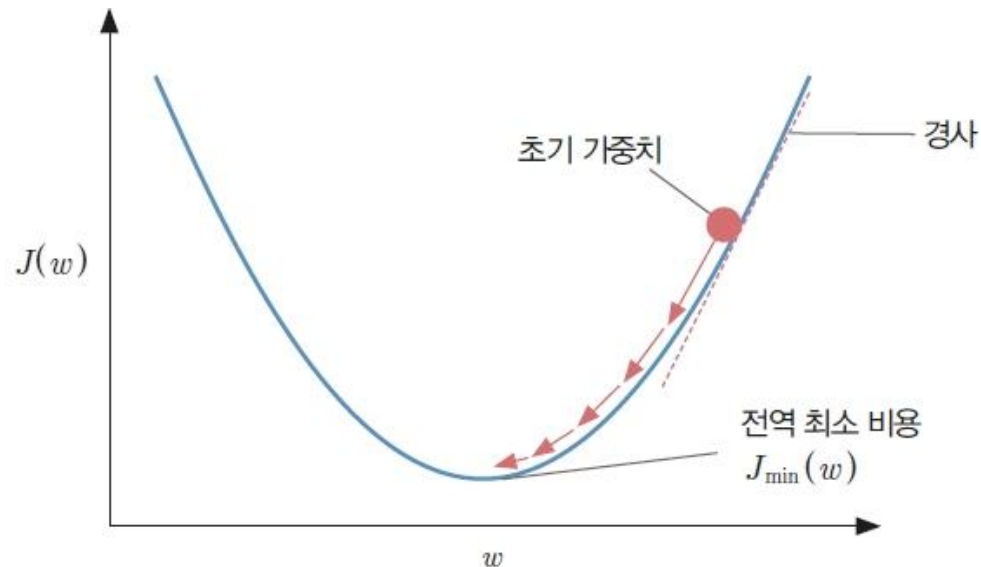


그림 7-6 $y = x^2$ 그래프에 경사하강법 적용

- 점이 최솟값을 달성하는 방향으로 점점 내려감
 - 몇 번 적용할 것인가? : 많이 실행할수록 최솟값에 가까워짐
 - 한 번에 얼마나 많이 내려갈 것인가? : 한 번에 얼마나 많은 공간을 움직일지를 기울기, 즉 경사라고 부름
 - 경사(gradient) : 경사하강법의 하이퍼 매개변수

04 경사하강법으로 선형회귀 풀기

2. 경사하강법 알고리즘

- $f(x)$ 는 최소화시켜야 하는 값이고,
 $2x$ 는 이를 미분한 값인 경사

$$f(x) = x^2 \rightarrow \frac{dy}{dx} = 2x$$

- 경사하강법의 기본 수식

$$x_{new} = x_{old} - \alpha \times (2x_{old})$$

- x_{old} 는 현재의 x 값, x_{new} 는 경사 값이 적용된 후 생성된 값
- 경사만큼의 변화가 계속 x 에 적용되어 x 의 최솟값 찾음
 - 반복적으로 미분 값을 적용시키면서 더 이상 값이 변하지 않거나 변화가 미미해지는 지점까지 값이 줄어든다

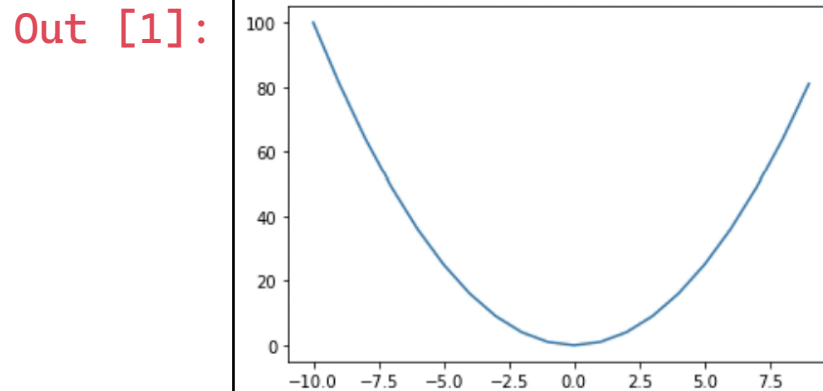
04 경사하강법으로 선형회귀 풀기

- 경사하강법을 코드로 표현

```
In [1]: import numpy as np
import matplotlib.pyplot as plt

x= np.arange(-10,10,1)
f_x = x ** 2

plt.plot(x, f_x)
plt.show()
```

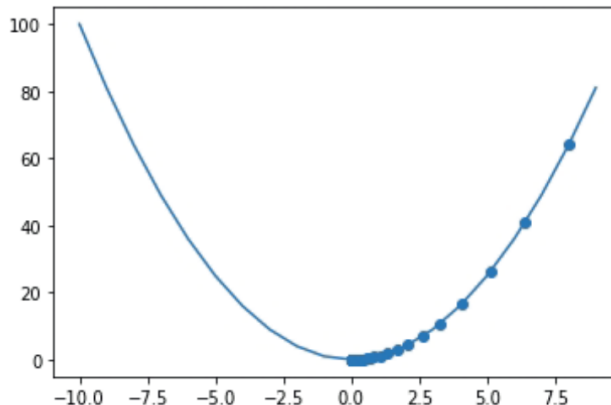


04 경사하강법으로 선형회귀 풀기

In [2]:

```
x_new = 10
derivative = []
y = []
learnng_rate= 0.1
for i in range(100):
    old_value = x_new
    derivative.append(old_value - learnng_rate * 2 *
old_value)
    x_new = old_value - learnng_rate *2* old_value
    y.append(x_new ** 2)
plt.plot(x, f_x)
plt.scatter(derivative, y)
plt.show()
```

Out [2]:



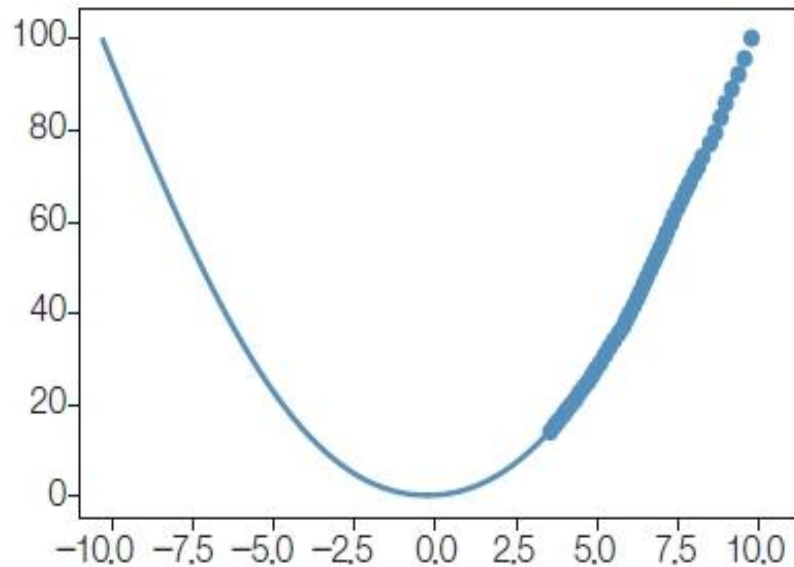
04 경사하강법으로 선형회귀 풀기

- 경사하강법에서 개발자가 결정해야 할 것
 - 학습률(learning rate)을 얼마나 할 것인가? α 값을 결정

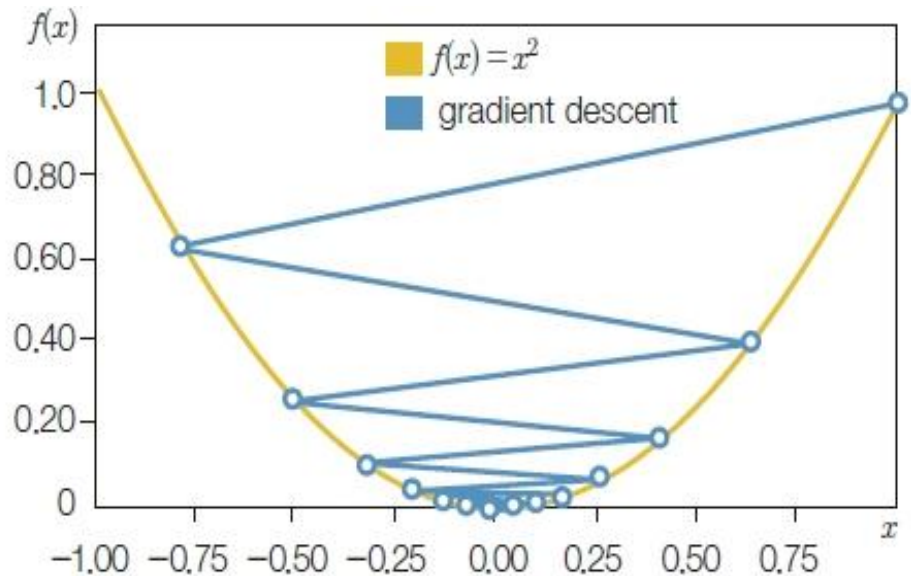
$$x_{new} = x_{old} - \alpha \times (2x_{old})$$

- 반복이 수행될 때마다 최솟값 변화
- 값이 너무 작으면 충분히 많은 반복을 적용해도 원하는 최적값을 찾지 못하는 경우 발생
- 값이 너무 크면 발산하여 최솟값 수렴 않거나 시간이 너무 오래 걸림
- 얼마나 많은 반복(iteration)으로 돌릴 것인가?
 - 반복 횟수가 충분하지 않다면 최솟값을 찾지 못하는 경우 발생
 - 반복 횟수가 너무 많다면 필요 없는 시간을 허비할 수도 있음

04 경사하강법으로 선형회귀 풀기



(a) 학습률이 작아 최솟값을 수렴하지 못하는 경우



(b) 학습률이 커서 발산한 경우

그림 7-7 학습률이 너무 크거나 작을 때 발생하는 문제

04 경사하강법으로 선형회귀 풀기

3. 경사하강법으로 선형회귀 풀기

- J 를 최소화하는 방향으로 학습을 실행하고 그 때의 w_0 와 w_1 을 찾기

$$J(w_0, w_1) = \frac{1}{2m} \sum_{i=1}^m (w_1 x^{(i)} + w_0 - y^{(i)})^2$$

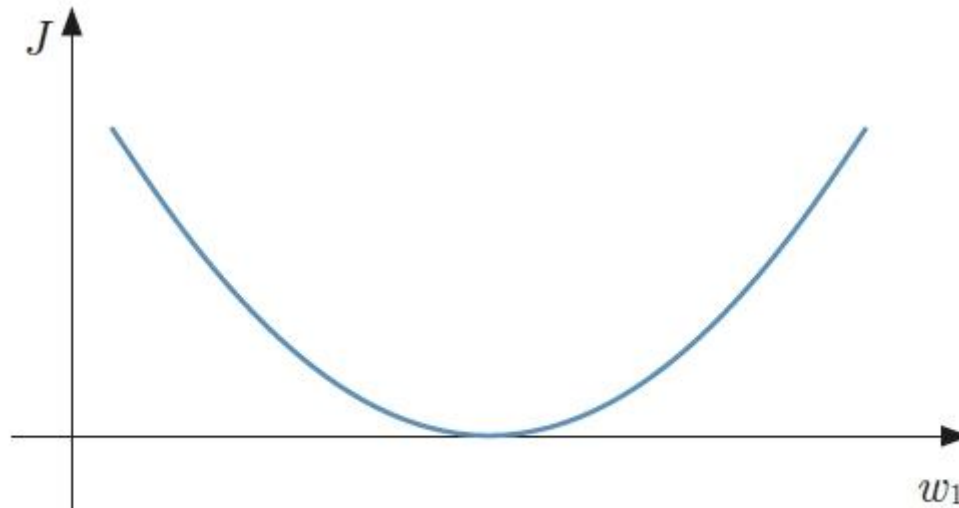


그림 7-8 $J(w_0, w_1)$ 그래프의 최적화

04 경사하강법으로 선형회귀 풀기

- w 변수가 두 개이기 때문에 3차원 그래프로 J 를 표현

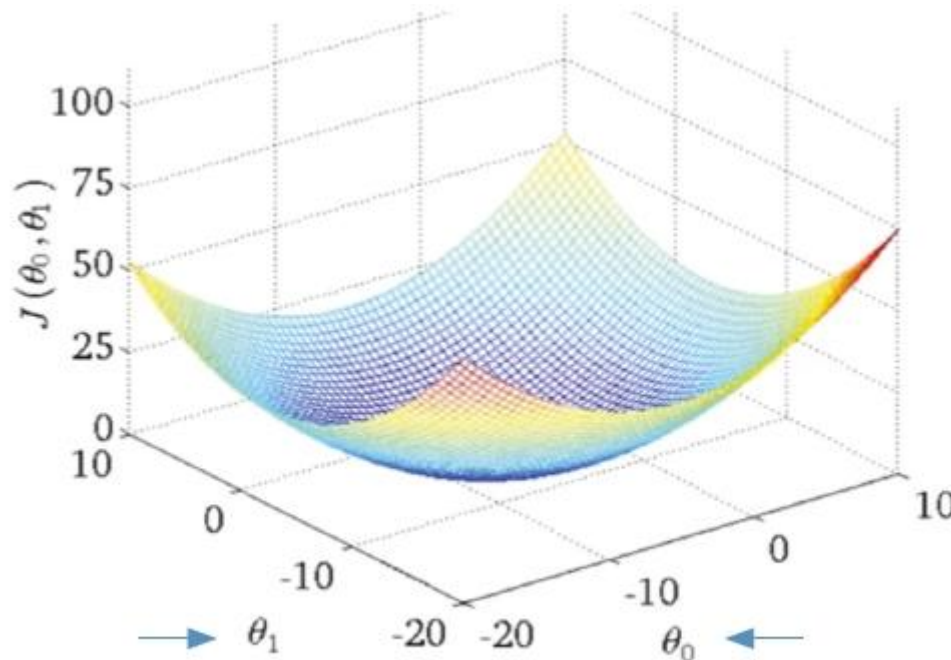


그림 7-9 비용함수 J 그래프 © Andrew Ng

04 경사하강법으로 선형회귀 풀기

- 매개변수 w_0 와 w_1 에 대한 업데이트
 - 먼저 임의의 w_0 와 w_1 을 지정
 - 비용함수 J 가 최소화 될 때까지 반복을 수행시켜 w_0 와 w_1 을 학습시킴
 - 비용함수가 더 이상 줄어들지 않거나 정해진 반복이 종료되면 학습이 끝남

```
loop until convergence {  
do  $\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1)$   
}
```

$$\frac{\partial J}{\partial w_0} = \frac{1}{m} \sum_{i=1}^m (w_1 x^{(i)} + w_0 - y^{(i)})$$

$$\frac{\partial J}{\partial w_1} = \frac{1}{m} \sum_{i=1}^m (w_1 x^{(i)} + w_0 - y^{(i)}) x^{(i)}$$

- w_0 와 w_1 업데이트는 동시에(simultaneously) 일어나 하나의 반복 안에서는 서로의 값에 영향을 주지 않음

05

선형회귀 성능 측정하기

05 선형회귀 성능 측정하기

1. 훈련/테스트 분할

- 훈련/테스트 분할(train/test split) : 머신러닝에서 데이터를 학습을 하기 위한 학습 데이터셋(train dataset)과 학습의 결과로 생성된 모델의 성능을 평가하기 위한 테스트 데이터셋(test dataset)으로 나눔
- 모델이 새로운 데이터셋에도 일반화(generalize)하여 처리할 수 있는지를 확인

05 선형회귀 성능 측정하기

- 홀드아웃 메서드(hold-out method) : 전체 데이터셋에서 일부를 학습 데이터와 테스트 데이터로 나누는 일반적인 데이터 분할 기법
 - 전체 데이터에서 랜덤하게 학습 데이터셋과 테스트 데이터셋을 나눔
 - 일반적으로 7:3 또는 8:2 정도의 비율

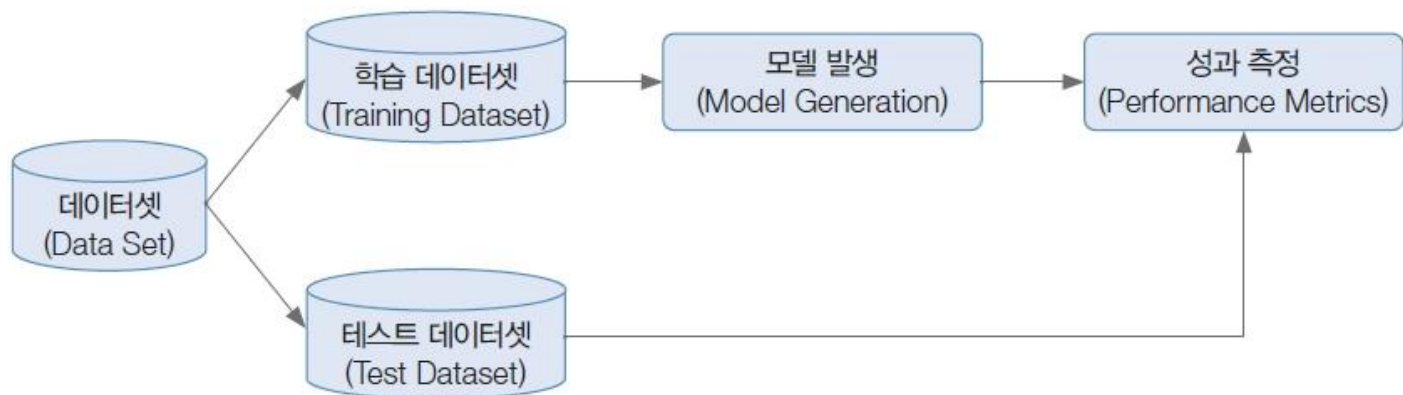


그림 7-11 홀드아웃 메서드(hold-out method) 기법

05 선형회귀 성능 측정하기

- sklearn 모듈이 제공하는 train_test_split 함수 사용
 - x와 y 벡터 값을 각각 넣고
 - 매개변수 test_size에 테스트 데이터로 사용할 데이터의 비율을 지정
 - random_state는 랜덤한 값을 기준으로 임의로 지정하는 값

```
In [1]: import numpy as np
        from sklearn.model_selection import train_test_split

        X, y = np.arange(10).reshape((5, 2)), range(5)

        X_train, X_test, y_train, y_test = train_test_split(
            X, y, test_size=0.33, random_state=42)
```

05 선형회귀 성능 측정하기

2. 선형회귀의 성능 측정 지표

2.1 MAE

- MAE(Mean Absolute Error) : 평균 절대 잔차
- 모든 테스트 데이터에 대해 예측값과 실제값의 차이에 대해 절대값을 구하고, 이 값을 모두 더한 후에 데이터의 개수만큼 나눈 결과

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| = \frac{1}{n} \sum_{i=1}^n |e_i|$$

- 직관적으로 예측값과 실측값의 차이를 알 수 있음

05 선형회귀 성능 측정하기

- sklearn 모듈에서는 median_absolute_error 함수로 MAE를 구함

In [2]:	<pre>from sklearn.metrics import median_absolute_error y_true = [3, -0.5, 2, 7] y_pred = [2.5, 0.0, 2, 8] median_absolute_error(y_true, y_pred)</pre>
Out [2]:	0.5

05 선형회귀 성능 측정하기

2.2 RMSE

- RMSE(Root Mean Squared Error) : 평균제곱근 오차
- 오차에 대해 제곱을 한 다음 모든 값을 더하여 평균을 낸 후 제곱근을 구함
- MAE에 비해 상대적으로 값의 차이가 더 큼
- 차이가 크게 나는 값에 대해서 페널티를 주고 싶다면 RMSE 값을 사용

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

05 선형회귀 성능 측정하기

- sklearn 모듈에서 RMSE를 직접적으로 지원하지는 않고 mean_squared_error만 지원

In [3]:	<pre>from sklearn.metrics import mean_squared_error y_true = [3, -0.5, 2, 7] y_pred = [2.5, 0.0, 2, 8] mean_squared_error(y_true, y_pred)</pre>
Out [3]:	0.375

05 선형회귀 성능 측정하기

2.3 결정계수

- 결정계수(R-squared) : 두 개의 값의 증감이 얼마나 일관성을 가지는지 나타내는 지표
- 예측값이 크면 클수록 실제값도 커지고, 예측값이 작으면 실제값도 작아짐
- 두 개의 모델 중 어떤 모델이 조금 더 상관성이 있는지를 나타낼 수 있지만, 값의 차이 정도가 얼마인지는 나타낼 수 없다는 한계가 있음

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \mu)^2}$$

05 선형회귀 성능 측정하기

- sklearn 모듈에서 r2_score 사용

In [4]:	<pre>from sklearn.metrics import r2_score y_true = [3, -0.5, 2, 7] y_pred = [2.5, 0.0, 2, 8] r2_score(y_true, y_pred)</pre>
Out [4]:	0.9486081370449679

Thank you!